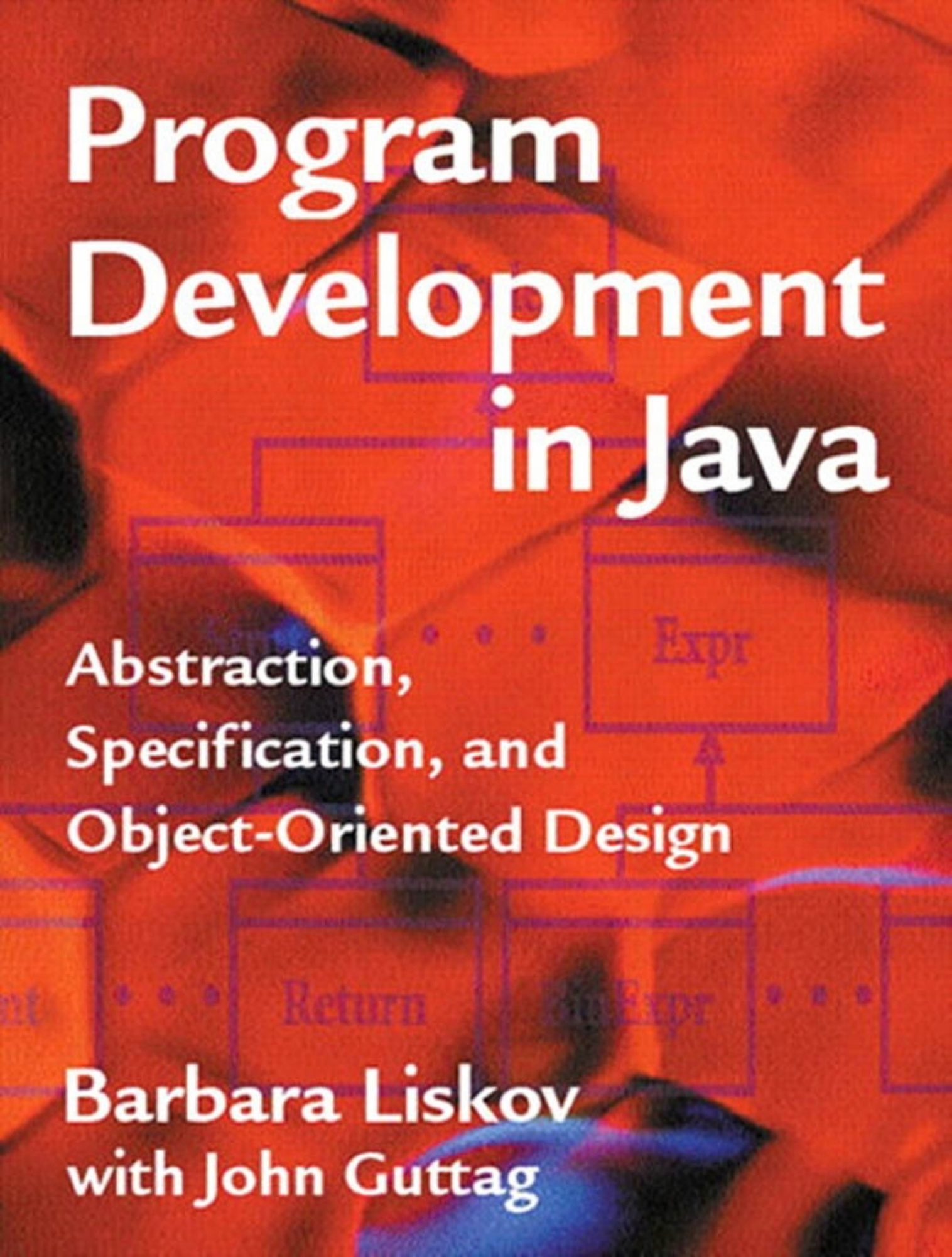
Program Development in Java

Abstraction, Specification, and Object-Oriented Design



Barbara Liskov
with John Guttag

Program Development in Java

This page intentionally left blank

Program Development in Java

Abstraction, Specification, and Object-Oriented Design

Barbara Liskov

with John Guttag

Addison-Wesley

Boston • San Francisco • New York • Toronto • Montreal
London • Munich • Paris • Madrid
Capetown • Sydney • Tokyo • Singapore • Mexico City

Pearson Education Corporate Sales Division
One Lake Street
Upper Saddle River, NJ 07458
(800) 382-3419
corpsales@pearsontechgroup.com

Visit us on the Web at *www.awl.com/cseng/*

Library of Congress Cataloging-in-Publication Data

Liskov, B.
Program development in Java : abstraction, specification, and object-oriented design / Barbara Liskov and John Guttag.
p. cm.
ISBN 0-201-65768-6 (alk. paper)
1. Java (Computer program language) 2. Object-oriented programming (Computer science) I. Guttag, John. II. Title.
QA76.73.J38 L58 2000
005.13′3—dc21 00-036277

ISBN 0201657686

Text printed on recycled and acid-free paper.
9 1011121314 CRW 08 07 06
9th Printing January 2006

To Nate and Moses

This page intentionally left blank

Contents

Preface

Constructing production-quality programs—programs that are used over an extended period of time—is well known to be extremely difficult. The goal of this book is to improve the effectiveness of programmers in carrying out this task. I hope the reader will become a better programmer as a result of reading the book. I believe the book succeeds at improving programming skills because my students tell me that it happens for them.

What makes a good programmer? It is a matter of efficiency over the entire production of a program. The key is to reduce wasted effort at each stage. Things that can help include thinking through your implementation before you start coding, coding in a way that eliminates errors before you test, doing rigorous testing so that errors are found early, and paying careful attention to modularity so that when errors are discovered, they can be corrected with minimal impact on the program as a whole. This book covers techniques in all these areas.

Modularity is the key to writing good programs. It is essential to break up a program into small modules, each of which interacts with the others through a narrow, well-defined interface. With modularity, an error in one part of a program can be corrected without having to consider all the rest of the code, and a part of the program can be understood without having to understand the entire thing. Without modularity, a program is a large collection of intricately interrelated parts. It is difficult to comprehend and to modify such a program, and also difficult to get it to work correctly.

The focus of this book therefore is on modular program construction: how to organize a program as a collection of well-chosen modules. The book relates modularity to abstraction. Each module corresponds to an abstraction, such as an index that keeps track of interesting words in a large collection of documents or a procedure that uses the index to find documents that

match a particular query. Particular emphasis is placed on object-oriented programming—the use of data abstraction and objects in developing programs.

The book uses Java for its programming examples. Familiarity with Java is not assumed. It is worth noting, however, that the concepts in this book are language independent and can be used to write programs in any programming language.

How Can the Book Be Used?

Program Development in Java can be used in two ways. The first is as the text for a course that focuses on an object-oriented methodology for the design and implementation of complex systems. The second is use by computing professionals who want to improve their programming skills and their knowledge of modular, object-oriented design.

When used as a text, the book is intended for a second or third programming course; we have used the book for many years in the second programming course at MIT, which is taken by sophomores and juniors. At this stage, students already know how to write small programs. The course builds on this material in two ways: by getting them to think more carefully about small programs, and by teaching them how to construct large programs using smaller ones as components. This book could also be used later in the curriculum, for example, in a software engineering course.

A course based on the book is suitable for all computer science majors. Even though many students will never be designers of truly large programs, they may work at development organizations where they will be responsible for the design and implementation of subsystems that must fit into the overall structure. The material on modular design is central to this kind of a task. It is equally important for those who take on larger design tasks.

What Is This Book About?

Roughly two-thirds of the book is devoted to the issues that arise in building individual program modules. The remainder of the book is concerned with how to use these modules to construct large programs.

Program Modules

This part of the book focuses on abstraction mechanisms. It discusses procedures and exceptions, data abstraction, iteration abstraction, families of data abstractions, and polymorphic abstractions.

Three activities are emphasized in the discussion of abstractions. The first is deciding on exactly what the abstraction is: what behavior it is providing to its users. Inventing abstractions is a key part of design, and the book discusses how to choose among possible alternatives and what goes into inventing good abstractions.

The second activity is capturing the meaning of an abstraction by giving a specification for it. Without some description, an abstraction is too vague to be useful. The specification provides the needed description. This book defines a format for specifications, discusses the properties of a good specification, and provides many examples.

The third activity is implementing abstractions. The book discusses how to design an implementation and the trade-off between simplicity and performance. It emphasizes encapsulation and the need for an implementation to provide the behavior defined by the specification. It also presents techniques—in particular, the use of representation invariants and abstraction functions—that help readers of code to understand and reason about it. Both rep invariants and abstraction functions are implemented to the extent possible, which is useful for debugging and testing.

The material on type hierarchy focuses on its use as an abstraction technique—a way of grouping related data abstractions into families. An important issue here is whether it is appropriate to define one type to be a subtype of another. The book defines the substitution principle—a methodical way for deciding whether the subtype relation holds by examining the specifications of the subtype and the supertype.

This book also covers debugging and testing. It discusses how to come up with a sufficient number of test cases for thorough black box and glass box tests, and it emphasizes the importance of regression testing.

Programming in the Large

The latter part of *Program Development in Java* is concerned with how to design and implement large programs in a modular way. It builds on the material about abstractions and specifications covered in the earlier part of the book.

The material on programming in the large covers four main topics. The first concerns requirements analysis—how to develop an understanding of what is wanted of the program. The book discusses how to carry out requirements analysis and also describes a way of writing the resulting requirements specification, by making use of a data model that describes the abstract state of the program. Using the model leads to a more precise specification, and it also makes the requirements analysis more rigorous, resulting in a better understanding of the requirements.

The second programming in the large topic is program design, which is treated as an iterative process. The design process is organized around discovering useful abstractions, ones that can serve as desirable building blocks within the program as a whole. These abstractions are carefully specified during design so that when the program is implemented, the modules that implement the abstractions can be developed independently. The design is documented by a design notebook, which includes a module dependency diagram that describes the program structure.

The third topic is implementation and testing. The book discusses the need for design analysis prior to implementation and how design reviews can be carried out. It also discusses implementation and testing order. This section compares top-down and bottom-up organizations, discusses the use of drivers and stubs, and emphasizes the need to develop an ordering strategy prior to implementation that meets the needs of the development organization and its clients.

This book concludes with a chapter on design patterns. Some patterns are introduced in earlier chapters; for example, iteration abstraction is a major component of the methodology. The final chapter discusses patterns not covered earlier. It is intended as an introduction to this material. The interested reader can then go on to read more complete discussions contained in other books.

Barbara Liskov

Acknowledgments

John Guttag was a coauthor of an earlier version of this book. Many chapters still bear his stamp. In addition, he has made numerous helpful suggestions about the current material.

Thousands of students have used various drafts of the book, and many of them have contributed useful comments. Scores of graduate students have been teaching assistants in courses based on the material in this book. Many students have contributed to examples and exercises that have found their way into this text. I sincerely thank all of them for their contributions.

My colleagues both at MIT and elsewhere have also contributed in important ways. Special thanks are due to Jeannette Wing and Daniel Jackson. Jeannette Wing (CMU) helped to develop the material on the substitution principle. Daniel Jackson (MIT) collaborated on teaching recent versions of the course and contributed to the material in many ways; the most important of these is the data model used to write requirements specifications, which is based on his research.

In addition, the publisher obtained a number of helpful reviews, and I want to acknowledge the efforts of James M. Coggins (University of North Carolina), David H. Hutchens (Millersville University), Gail Kaiser (Columbia University), Gail Murphy (University of British Columbia), James Purtilo (University of Maryland), and David Riley (University of Wisconsin at LaCrosse). I found their comments very useful, and I tried to work their suggestions into the final manuscript.

Finally, MIT's Department of Electrical Engineering and Computer Science and its Laboratory for Computer Science have supported this project in important ways. By reducing my teaching load, the department has given me time to write. The laboratory has provided an environment that enabled research leading to many of the ideas presented in this book.

This page intentionally left blank

Introduction

1

This book will develop a methodology for constructing software systems. Our goal is to help programmers construct programs of high quality—programs that are reliable, efficient, and reasonably easy to understand, modify, and maintain.

A very small program, consisting of no more than a few hundred lines, can be implemented as a single monolithic unit. As the size of the program increases, however, such a monolithic structure is no longer reasonable because the code becomes difficult to understand. Instead, the program must be decomposed into a number of small independent programs, called *modules*, that together provide the desired function. We shall focus on this decomposition process: how to decompose large programming problems into small ones, what kinds of modules are most useful in this process, and what techniques increase the likelihood that modules can be combined to solve the original problem.

Doing decomposition properly becomes more and more important as the size of the program increases for a number of reasons. First, many people must be involved in the construction of a large program. If just a few people are working on a program, they naturally interact regularly. Such contact reduces the possibility of misunderstandings about who is doing what and lessens the seriousness of the consequences should misunderstandings occur. If many people work on a project, regular communication becomes impossible because

it consumes too much time. Instead, the program must be decomposed into pieces that the individuals can work on independently with a minimum of contact.

The useful life of a program (its *production* phase) begins when it is delivered to the customer. Work on the program is not over at this point, however. The code will probably contain residual errors that will need attention, and program modifications will often be required to upgrade the program's serviceability or to provide services better matched to the user's needs. This activity of program *modification* and *maintenance* is likely to consume more than half of the total effort put into the project.

For modification and maintenance, it is rarely practical to start from scratch and reimplement the entire program. Instead, one must retrofit modifications within the existing structure, and it is therefore important that the structure accommodate change. In particular, the pieces of the program must be independent, so that a change to one piece can be made without requiring changes to all pieces.

Finally, most programs have a long lifetime. Programmers often have to deal with programs long after they have first worked on them. Moreover, there is likely to be substantial turnover of personnel over the life of any project, and program modification and maintenance are typically done by people other than the original implementors. All of these factors require that programs be structured in such a way that they can be understood easily.

In the methodology we shall describe in this book, programs will be developed by means of problem decomposition based on a recognition of useful abstractions. *Decomposition* and *abstraction*, the two key concepts in this book, form our next subject.

1.1 Decomposition and Abstraction

The basic paradigm for tackling any large problem is clear—we must "divide and rule." Unfortunately, merely deciding to follow Machiavelli's dictum still leaves us a long way from solving the problem at hand. Exactly how we choose to divide the problem is of overriding importance.

Our goal in decomposing a program is to create modules that are themselves small programs that interact with one another in simple, well-defined ways. If we achieve this goal, different people will be able to work on dif-

ferent modules independently, without needing much communication among themselves, and yet the modules will work together. In addition, during program modification and maintenance, it will be possible to modify some of the modules without affecting all of the others.

When we decompose a problem, we factor it into separable subproblems in such a way that

- Each subproblem is at the same level of detail.
- Each subproblem can be solved independently.
- The solutions to the subproblems can be combined to solve the original problem.

Sorting using merge sort is an elegant example of problem solving by decomposition. It breaks the problem of sorting a list of arbitrary size into the two simpler problems of sorting a list of size two and merging two sorted lists of arbitrary size.

Decomposition is a time-honored and useful technique in many disciplines. From Babbage's day onward, people have recognized the utility of such things as macros and subroutines as decomposition devices for programmers. It is important to recognize, however, that decomposition is not a panacea and when used improperly, it can have a harmful effect. Furthermore, large or poorly understood problems are difficult to decompose properly. The most common problem is creating individual components that solve the stated subproblems but do not combine to solve the original problem. This is one of the reasons why system integration is often difficult.

For example, imagine creating a play by assembling a group of writers, giving each a list of characters and a general plot outline, and asking each of them to write a single character's lines. The authors might accomplish their individual tasks admirably, but it is highly unlikely that their combined efforts will be an admirable play. It would probably lack any sort of coherence or sense. Individually acceptable solutions simply cannot be expected to combine properly if the original task has been divided in a counterproductive way.

Abstraction is a way to do decomposition productively by changing the level of detail to be considered. When we abstract from a problem, we agree to ignore certain details in an effort to convert the original problem to a simpler one. We might, for example, abstract from the problem of writing a play to the problem of deciding how many acts it should have, or what its plot will

be, or even the sense (but not the wording) of individual pieces of dialog. After this has been done, the original problem (of writing all of the dialog) remains, but it has been considerably simplified—perhaps even to the point where it could be turned over to another or even several others. (Alexandre Dumas *père* churned out novels in this way.)

The paradigm of abstracting and then decomposing is typical of the program design process: decomposition is used to break software into components that can be combined to solve the original problem; abstractions assist in making a good choice of components. We alternate between the two processes until we have reduced the original problem to a set of problems we already know how to solve.

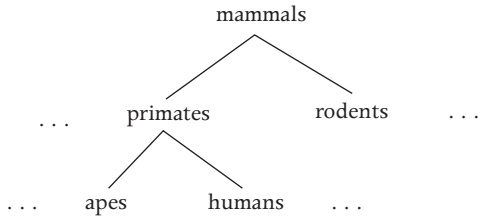
1.2 Abstraction

The process of abstraction can be seen as an application of a many-to-one mapping. It allows us to forget information and consequently to treat things that are different as if they were the same. We do this in the hope of simplifying our analysis by separating attributes that are relevant from those that are not. It is crucial to remember, however, that relevance often depends upon context. In the context of an elementary school classroom we learn to abstract both $(\frac{8}{3}) \times 3$ and $5 + 3$ to the concept we represent by the numeral 8. Much later we learn, often under unpleasant circumstances, that on many computing machines this abstraction can get us into a world of trouble.

For example, consider the structure shown in Figure 1.1, in which the concept is "mammal." All mammals share certain characteristics, such as the fact that females produce milk. At this level of abstraction, we focus on these common characteristics and ignore the differences between the various types of mammals.

At a lower level of abstraction, we might be interested in particular instances of mammals. However, even here we can abstract by considering not individuals, or even species, but groups of related species. At this level, we would have groupings such as primates or rodents. Here again, we are interested in common characteristics rather than the differences between, say, humans and chimpanzees. Such differences are relevant at a still lower level of abstraction.

Figure 1.1 An abstraction hierarchy



The abstraction hierarchy of Figure 1.1 comes from the field of zoology, but it might well appear in a program that implemented some zoological application. A more specifically computer-oriented example that is useful in many programs is the concept of a "file." Files abstract from raw storage and provide long-term, online storage of named entities. Operating systems differ in their realizations of files; for example, the structure of the filenames differs from system to system, as does the way in which the files are stored on secondary storage devices.

In this book, we are interested in abstraction as it is used in programs in general. The most significant development to date in this area is high-level languages. By dealing directly with the constructs of a high-level language, rather than with the many possible sequences of machine instructions into which they can be translated, the programmer achieves a significant simplification.

In recent years, however, programmers have become dissatisfied with the level of abstraction generally achieved even in high-level language programs. Consider, for example, the program fragments in Figure 1.2. At the level of abstraction defined by the programming language, these fragments are clearly different: if there is an occurrence of `e` in `a`, one fragment finds the index of the first occurrence and the other, the index of the last. If `e` does not occur in `a`, one sets `i` to `a.length` and the other to -1. It is not improbable, however, that both were written to accomplish the same goal: to set `found` to false if there is no occurrence of `e` in `a` and, otherwise, to set `found` to true and `z` to the index of some occurrence of `e` in `a`. If this is what we want, it is not evident from the program fragments by themselves.

One approach to dealing with this problem lies in the invention of "very-high-level languages" built around some fixed set of relatively general data structures and a powerful set of primitives that can be used to manipulate

Figure 1.2 Two program fragments

```
// search upwards
found = false;
for (int i = 0; i < a.length; i++)
   if (a[i] == e) {
      z = i;
      found = true;
   }

// search downwards
found = false;
for (int i = a.length-1; i >= 0; i--)
   if (a[i] == e) {
      z = i;
      found = true;
   }
```

them. For example, suppose a language provided `isIn` and `indexOf` as primitive operations on arrays. Then we could accomplish the task outlined in Figure 1.2 simply by writing

```
found = a.isIn(e);
if (found)
    z = a.indexOf(e);
```

The flaw in this approach is that it presumes that the designer of the programming language will build into the language most of the abstractions that users of the language will want. Such foresight is not given to many; and even if it were, a language containing so many built-in abstractions might well be so unwieldy as to be unusable.

A preferable alternative is to design into the language mechanisms that allow programmers to construct their own abstractions as they need them. One common mechanism is the use of *procedures*. By separating procedure definition and invocation, a programming language makes two important methods of abstraction possible: *abstraction by parameterization* and *abstraction by specification*. These abstraction mechanisms are summarized in Sidebar 1.1.

Sidebar 1.1 Abstraction Mechanisms

- *Abstraction by parameterization* abstracts from the identity of the data by replacing them with parameters. It generalizes modules so that they can be used in more situations.
- *Abstraction by specification* abstracts from the implementation details (how the module is implemented) to the behavior users can depend on (what the module does). It isolates modules from one another's implementations; we require only that a module's implementation supports the behavior being relied on.

1.2.1 Abstraction by Parameterization

Abstraction by parameterization, through the introduction of parameters, allows us to represent a potentially infinite set of different computations with a single program text that is an abstraction of all of them. For example,

$$x * x + y * y$$

describes a computation that adds the square of the value stored in the variable x to the square of the value stored in the variable y.

On the other hand, the *lambda expression*

$$\lambda\, x, y\text{: int.}(x * x + y * y)$$

describes the set of computations that square the value stored in some integer variable, which we shall temporarily refer to as x, and add to it the square of the value stored in another integer variable, which we shall temporarily call y. In such a lambda expression, we refer to x and y as the *formal parameters* and $x * x + y * y$ as the *body* of the expression. We invoke a computation by binding the formal parameters to arguments and then evaluating the body. For example,

$$\lambda\, x, y\text{: int.}(x * x + y * y)(w, z)$$

is identical in meaning to

$$w * w + z * z$$

In more familiar notation, we might denote the previous lambda expression by

```
int squares (int x, y) {
   return x * x + y * y;
}
```

and the binding of actual to formal parameters and evaluation of the body by the procedure call

```
u = squares(w, z);
```

Programmers often use abstraction by parameterization without even noticing that they are doing so. For example, suppose we need a procedure that sorts an array of integers `a`. At some time in the future, we shall probably have to sort some other array, perhaps even somewhere else in this same program. It is highly unlikely, however, that every array we need to sort will be named `a`; we therefore invoke abstraction by parameterization to generalize the procedure and thus make it more useful.

Abstraction by parameterization is an important means of achieving generality in programs. A sort routine that works on any array of integers is much more generally useful than one that works only on a particular array of integers. By further abstraction, we can achieve even more generality. For example, we might define a `sort` abstraction that works on arrays of reals as well as arrays of integers, or even one that works on arraylike structures in general.

Abstraction by parameterization is an extremely powerful mechanism. Not only does it allow us to describe a large (even infinite) number of computations relatively simply, but it is easily and efficiently realizable in programming languages. Nonetheless, it is not a sufficiently powerful mechanism to describe conveniently and fully the abstraction that the careful use of procedures can provide.

1.2.2 Abstraction by Specification

Abstraction by specification allows us to abstract from the computation (or computations) described by the body of a procedure to the end that procedure was designed to accomplish. We do this by associating with each procedure a *specification* of its intended effect and then considering the meaning of a

Figure 1.3 The `sqrt` procedure

```
float sqrt (float coef) {
   // REQUIRES: coef > 0
   // EFFECTS: Returns an approximation to the square root of coef
   float ans = coef/2.0;
   int i = 1;
   while (i < 7) {
      ans = ans - ((ans * ans - coef)/(2.0*ans));
      i = i + 1;
   }
   return ans;
}
```

procedure call to be based on this specification rather than on the procedure's body.

We are making use of abstraction by specification whenever we associate with a procedure a comment that is sufficiently informative to allow others to use that procedure without looking at its body. A good way to write such comments is to use pairs of *assertions*. The *requires assertion* (or *precondition*) of a procedure specifies something that is assumed to be true on entry to the procedure. In practice, what is most often asserted is a set of conditions sufficient to ensure the proper operation of the procedure. (This is often simply the vacuous assertion "true.") The *effects assertion* (or *postcondition*) specifies something that is supposed to be true at the completion of any invocation of the procedure for which the precondition was satisfied.

Consider, for example, the `sqrt` procedure in Figure 1.3. Because a specification is provided, we can ignore the body of the procedure and take the meaning of the procedure call `y = sqrt(x)` to be "If `x` is greater than zero when the procedure is invoked, then after the execution of the procedure, `y` is an approximation to the square root of `x`." Notice that the requires and effects assertions permit us to say nothing about the value of `y` if `x` is not greater than zero. This is important, since a user might otherwise quite reasonably assume that `sqrt(0)` returned a meaningful answer.

In using a specification to reason about the meaning of a procedure call, we follow two distinct rules:

1. After the execution of the procedure, we can assume that the postcondition holds provided the precondition held when the call was made.

2. We can assume *only* those properties that can be inferred from the postcondition.

The two rules mirror the two benefits of abstraction by specification. The first asserts that users of the procedure need not bother looking at the body of the procedure in order to use it. They are thus spared the effort of first understanding the details of the computations described by the body and then abstracting from these details to discover that the procedure really does compute an approximation to the square root of its argument. For complicated procedures, or even simple ones using unfamiliar algorithms, this is a nontrivial benefit.

The second rule makes it clear that we are indeed abstracting from the procedure body, that is, omitting some supposedly irrelevant information. This insistence on forgetting information is what distinguishes abstraction from decomposition. By examining the body of `sqrt`, users of the procedure could gain a considerable amount of information that cannot be gleaned from the postcondition and therefore should not be relied on—for example, that `sqrt`(4) will return +2. In the specification, however, we are saying that this information about the returned result is to be ignored. We are thus saying that the procedure `sqrt` is an abstraction representing the set of all computations that return "an approximation to the square root of x."

In this book, abstraction by specification will be the major method used in program construction. Abstraction by parameterization will be taken almost for granted; abstractions will have parameters as a matter of course.

1.2.3 Kinds of Abstractions

Abstraction by parameterization and by specification are powerful methods for program construction. They enable us to define three different kinds of abstractions: procedural abstraction, data abstraction, and iteration abstraction. In general, each procedural, data, and iteration abstraction will incorporate both methods within it.

For example, `sqrt` is like an operation: it abstracts a single action or task. We shall refer to abstractions that are operationlike as *procedural abstractions*. Note that `sqrt` incorporates both abstraction by parameterization and abstraction by specification.

Procedural abstraction is a powerful tool. It allows us to extend the virtual machine defined by a programming language by adding a new operation. This kind of extension is most useful when we are dealing with problems that are

conveniently decomposable into independent functional units. However, it is often more fruitful to think of adding new kinds of data objects to the virtual machine.

The behavior of the data objects is expressed most naturally in terms of a set of operations that are meaningful for those objects. This set includes operations to create objects, to obtain information from them, and possibly to modify them. For example, `push` and `pop` are among the meaningful operations for stacks, integers need the usual arithmetic operations, and a bank account object in a banking system would have operations to deposit and withdraw money. Thus a *data abstraction* (or *data type*) consists of a set of objects and a set of operations characterizing the behavior of the objects.

As an example, consider `MultiSets`. `MultiSets` are like ordinary sets except that elements can occur more than once in a `MultiSet`. `MultiSet` operations might include `empty`, `insert`, `delete`, `numberOf`, and `size`. These operations create an empty `MultiSet`, add and delete elements from a `MultiSet`, tell how many times a particular element occurs in a `MultiSet`, and tell how many elements are in a `MultiSet`, respectively. The operations might be implemented within the runtime environment of the programming language by calls to various procedures. Programmers using `MultiSets`, however, need not worry about how these procedures are implemented. To them `empty`, `insert`, `delete`, `numberOf`, and `size` are abstractions defined by such statements as

- The `size` of the `MultiSet` `insert(s, e)` is equal to `size(s) + 1`.
- For all `e`, the `numberOf` times `e` occurs in the `MultiSet` `empty( )` is 0.

The key thing to notice is that each of these statements deals with more than one operation. We do not present independent definitions of each operation, but rather define them by showing how they relate to one another. The emphasis on the relationships among operations is what makes a data abstraction something more than just a set of procedures. The importance of this distinction is discussed throughout this book.

In addition to procedural and data abstraction, we shall also deal with *iteration* abstraction. Iteration abstraction is used to avoid having to say more than is relevant about the flow of control in a loop. A typical iteration abstraction might allow us to iterate over all the elements of a `MultiSet` without constraining the order in which the elements are to be processed.

Sidebar 1.2 Kinds of Abstractions

- *Procedural abstraction* allows us to introduce new operations.
- *Data abstraction* allows us to introduce new types of data objects.
- *Iteration abstraction* allows us to iterate over items in a collection without revealing details of how the items are obtained.
- *Type hierarchy* allows us to abstract from individual data types to families of related types.

Finally, we shall sometimes abstract groups of data abstractions into *type families*. All members of the family have operations in common; these common operations are defined in the *supertype*, the type that is the ancestor of all the others, which are its *subtypes*. For example, there might be a family of types that provide the ability to read from input streams. The supertype would provide basic operations to `open` a stream, to `read` a character or a string, and to `close` a stream. The subtypes will then provide additional operations, for example, allowing data to be streamed from a file. Type families abstract from the details that distinguish members of a family from one another, to their commonalities. They allow programmers to ignore the differences most of the time.

Sidebar 1.2 summarizes the kinds of abstractions.

1.3 The Remainder of the Book

This book is concerned with how to do program decomposition based on abstraction. Our emphasis will be on data abstraction. We believe that while procedural and iteration abstraction have valuable roles to play, it is data abstraction that most often provides the primary organizational tool in the programming process. Data abstraction is the basis of object-oriented programming and design.

However, before we can decompose intelligently, we need a thorough understanding of the kinds of abstractions we are aiming for. The next several chapters are concerned with this topic. Since we will be using Java as our im-

plementation mechanism, we begin by providing a brief introduction to Java and the way it supports object-oriented programming. Then we discuss the various kinds of abstractions—what they are, how to specify their behavior, and how to implement them in Java.

Our discussion shall emphasize how to get things right: how to develop a good abstraction, how to specify it clearly, and how to implement it correctly. Our programs ultimately will consist of many modules, each corresponding to the implementation of an abstraction. Unless each of these modules works individually as required, the program as a whole will not function properly. Therefore, understanding how to program at the module level—called *programming in the small*—is essential to achieving our ultimate goal of developing complete programs—called *programming in the large*.

The latter portion of the book focuses on programming in the large and in particular on the use of abstractions in program construction. We discuss the phases of program construction, how to do program design, and how to carry on into implementation. The book concludes with a discussion of a number of techniques, called *design patterns*, for organizing the structure of a program to improve its flexibility or performance.

Exercises

1.1 Describe an abstraction hierarchy with which you are familiar.

1.2 Select a procedure that you have written or used and discuss how it supports abstraction by specification and by parameterization.

This page intentionally left blank

Understanding Objects in Java

2

Java is an object-oriented language. This means that most of the data manipulated by programs are contained in *objects*. Objects contain both state and operations; the operations are called *methods*. Programs interact with objects by invoking their methods. The methods provide access to the state, allowing using code to observe the current state of an object or to modify it. Sidebar 2.1 provides information about the origins of Java.

This chapter provides some basic information about Java and its support for object-oriented programming and design. We shall concentrate primarily on the language *semantics*, that is, on the meaning of constructs in the language. For a complete, detailed description of the language, you should consult a Java text or reference manual. A Java reference manual is available online.

2.1 Program Structure

Java programs are made up of classes and interfaces. *Classes* are used in two different ways: to define collections of procedures (this use is discussed in Chapter 3) and to define new data types, such as `MultiSet` (this use is discussed in Chapter 5). *Interfaces* are also used to define new data types.

Sidebar 2.1 Origins of Java

- Java is a successor to a number of languages, including Lisp, Simula67, CLU, and SmallTalk.
- Java is superficially similar to C and C++ because its syntax is borrowed from them. However, at a deeper level it is very different from these languages.

The majority of the content of both classes and interfaces consists of definitions of *methods*. A class that defines a group of procedures provides a method for each procedure; for example, a class providing procedures that manipulate integer arrays might contain a method to sort an array and another method to search an array for a match with a particular integer. A class or interface that defines a data type provides methods for the operations associated with the objects of that type. For example, in the case of a `MultiSet`, there might be an `insert` method to add an integer in the `MultiSet` and a `numberOf` method to determine how many times a given integer appears in the `MultiSet`.

An example of a class that defines a group of procedures is given in Figure 2.1. (Comments begin with the `//` symbol and continue to the end of the line.) Such methods are named by indicating their class and then their method name. Here are examples of calls of the methods in class `Num`:

```
int x = Num.gcd(15, 6);
if (Num.isPrime(y)) ...
```

A method takes zero or more arguments and returns a single result. Its *header* indicates this information. The arguments are often referred to as the *formal parameters* or *formals* of the call. For example, `gcd` has two formals, `x` and `y`, both of which are integers; it returns an integer result. A method may also terminate by throwing an exception. Exceptions will be discussed in detail in Chapter 4.

Because Java requires that every method have a result, a special form is used when there is no result. Such a method indicates that its return type is `void`. For example, suppose the `Arrays` class provides routines that are useful in manipulating arrays, among them a way of sorting arrays:

Figure 2.1 The Num class

```
public class Num {
   // class providing useful numeric routines

   public static int gcd (int n, int d) {
      // REQUIRES: n and d to be greater than zero
      // the gcd is computed by repeated subtraction
      while (n != d)
         if (n > d) n = n - d; else d = d - n;
      return n;
   }

   public static boolean isPrime(int p) {
      // implementation goes here
   }
}
```

```
   public static void sort (int[ ] a)
```

(The form `int[ ]` indicates that the argument is an array of integers of unspecified length.) This method doesn't return a result; instead, it sorts its argument array in place. It can be called as follows:

```
   Arrays.sort(a); // a call; assume a is an array of ints
```

2.2 Packages

Classes and interfaces are grouped into *packages*. Packages serve two purposes. First, they are an *encapsulation* mechanism; they provide a way to share information within the package while preventing its use on the outside.

Each class and interface has a declared *visibility*. Only classes and interfaces declared to be *public* can be used by code in other packages—for example, the Num class in Figure 2.1 can be used outside its package. The remaining definitions can be used only within the package.

In addition, the declarations within a class have a declared visibility. Only entities declared to be public, such as the gcd and isPrime methods in the Num class, are accessible to code in other packages. Other kinds of declared

visibility limit the code that can access the entity—for example, to just its class or just its package; the details will be discussed in later chapters.

The other use of packages is for *naming*. Each package has a hierarchical name that distinguishes it from all other packages. Classes and interfaces within the package have names that are relative to the package name. This means that there are no name conflicts between classes and interfaces defined in different packages.

Code in a package can refer to other classes and interfaces of its own package by using their class or interface name. For example, if the `mathRoutines` package contains the class `Num`, code within that package can refer to that class by using the name `Num`. Definitions in other packages can be referred to using their *fully qualified names*—that is, their name appended to their package's hierarchical name. For example, the fully qualified name for the `Num` class might be `mathRoutines.Num`. It is also possible to use short names to refer to definitions in other packages by using the *import* statement, to either import all public definitions from a package, or to import specific public definitions from a package. In either case, the imported definition can be referred to using its class or interface name.

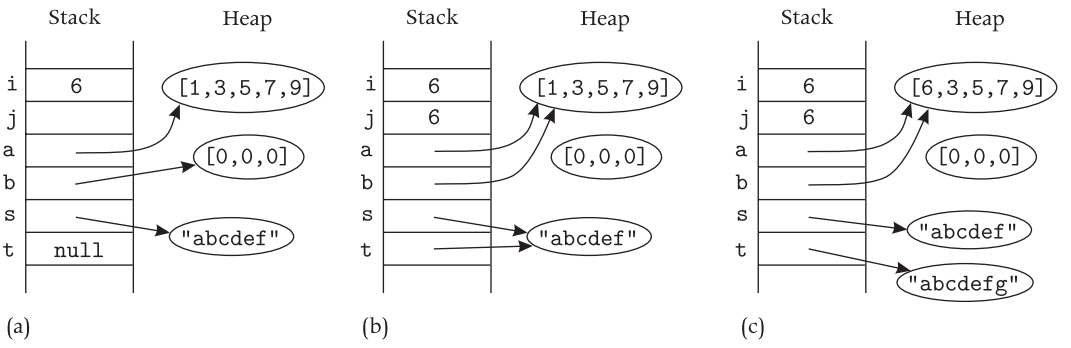
One problem with short names is the possibility of name conflicts. For example, suppose two packages both define classes, named `Num`. In this case, if code uses both classes, it cannot use a short name for each. Either it could use a fully qualified name for each or it could import one of the classes and use a long name for the other.

Sometimes there is a conflict between encapsulation and naming. It is convenient to group many definitions in the same package because then code outside the package has access to all of them by importing the whole package. But this kind of grouping may be wrong from the point of encapsulation because code within a package can sometimes access internal information of other definitions within that package. In general, such a conflict should be resolved in favor of encapsulation.

2.3 Objects and Variables

All data are accessed by means of *variables*. *Local variables*, such as those declared within methods, reside on the runtime *stack*; space is allocated for them when the method is called and deallocated when the method returns.

Figure 2.2 Objects and variables



Every variable has a declaration that indicates its *type*. *Primitive types*, such as `int` (integers), `boolean`, and `char` (characters), define sets of values, such as `3`, `false`, or `c`. All other types define sets of objects. Some object types will be provided for you, in packages defined by others. One such package is `java.lang`, which provides a number of useful types such as `String`; the types defined by this package can be used without importing the package. Others you will define yourselves. Chapter 5 describes how to define new types.

Variables of primitive types contain *values*. For example, the following:

```
int i = 6;
```

stores the value `6` in variable `i`. Variables of all other types, including strings and arrays, contain *references* to *objects* that reside on the *heap*. Objects are created on the heap by use of the `new` operator. Thus,

```
int [ ] a = new int[3];
```

causes space for a new array of integers object, with room for three integers, to be allocated on the heap and a reference to the new object to be stored in `a`.

Local variables can be initialized when they are declared. They *must* be initialized before their first use. Furthermore, if the compiler cannot prove that a variable is initialized before first use, it will cause compilation to fail.

Objects and variables are illustrated in Figure 2.2a, which shows the stack and heap after processing the following declarations:

```
int i = 6;
int j; // uninitialized
int [ ] a = {1,3,5,7,9}; // creates a 5-element array
int [ ] b = new int[3];
String s = "abcdef"; // creates a new string
String t = null;
```

Here all variables except `j` have been given an initial value. Variable `t` has been initialized to `null`; this special value provides a way of initializing a variable that will eventually refer to an object.

This example shows a number of object creations. Strings are created by indicating their content; thus, `s` refers to a string object containing `"abcdef"`. Arrays can be created similarly, by indicating their elements; thus, `a` refers to a five-element array. The assignment to `b` shows the usual way of creating a new object, by calling the built-in `new` operator. This operator creates an object of the indicated class on the heap and then initializes it by running a special kind of method, called a *constructor*, for that class. For example, the array constructor initializes each element of a new array of integers to `0`. Thus, `b` refers to a three-element array of integers, where each element of the array is `0`.

Every object has an identity that is distinct from that of every other object. That is, when an object is created by a call to `new`, or through use of the special forms such as `"abcdef"` for strings and `{1,3,5,7,9}` for arrays, what is obtained is an object that is distinct from any other object in existence.

An assignment

```
v = e;
```

copies the value obtained by evaluating the expression `e` into the variable `v`. If the expression evaluates to a reference to an object, the reference is copied. This situation is illustrated in Figure 2.2b, which shows the results of the following assignments:

```
j = i;
b = a;
t = s;
```

Note that in the case of the string and array variables, we now have two variables pointing to the same object. Thus, assignment involving references causes variables to *share* objects.

The == operator can be used to determine whether two variables contain the same value. This operator is used primarily for primitive types—for example, to compare two `int`s, as in `j == i`, or to determine whether a variable that might refer to an object instead contains `null`, such as `t == null`. It can also be used to determine whether two variables refer to the same object; in the situation in Figure 2.2a, for example, `a == b` will not be true, whereas in the situation in Figure 2.2b, `a == b` is true.

Objects in the heap continue to exist as long as they are reachable from some variable on the stack, either directly or via a path through other objects. When an object is no longer reachable, its storage becomes available for reclamation by the garbage collector. For example, in the state shown in Figure 2.2b, the array formerly referred to by `b` is no longer reachable and is therefore available for reclamation by the garbage collector.

2.3.1 Mutability

All objects are either *immutable* or *mutable*. The state of an immutable object never changes, while the state of a mutable object can change.

Strings are immutable: there are no `String` methods that cause the state of a `String` object to change. For example, strings have a concatenation operator `+`, but it does not modify either of its arguments; instead, it returns a new string whose state is the concatenation of the states of its arguments. If we did the following assignment to `t` with the state shown in Figure 2.2b:

```
t = t + "g";
```

the result as shown in Figure 2.2c is that `t` now refers to a new `String` object whose state is `"abcdefg"` and the object referred to by `s` is unaffected.

On the other hand, arrays are mutable. The assignment

```
a[i] = e;
```

causes the state of array `a` to change by replacing its i^{th} element with the value obtained by evaluating expression `e`. (The modification occurs only if `i` is in bounds for `a`; otherwise, an exception is thrown.)

If a mutable object is shared by two or more variables, modifications made through one of the variables will be visible when the object is used through the other variable. For example, suppose the shared array in Figure 2.2b is modified by

Sidebar 2.2 Mutability and Sharing

- An object is *mutable* if its state can change. For example, arrays are mutable.
- An object is *immutable* if its state never changes. For example, strings are immutable.
- An object is *shared* by two variables if it can be accessed through either of them.
- If a mutable object is shared by two variables, modifications made through one of the variables will be visible when the object is used through the other.

```
b[0] = i;
```

This causes the zeroth element of the array to contain 6 (instead of the 1 it used to contain), as shown in Figure 2.2c. Furthermore, the change is visible when the array is used later, via either variable `b` or variable `a`; for example, in

```
if (a[0] == i) ...
```

the expression will evaluate to true, and therefore, the then branch will be executed.

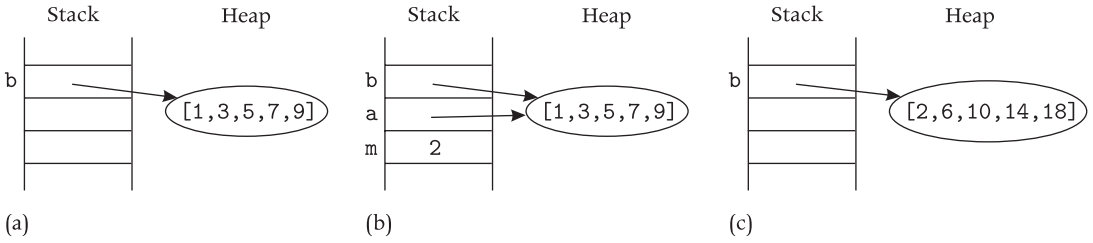
Sidebar 2.2 summarizes mutability and sharing.

2.3.2 Method Call Semantics

An attempt to call a method, `e.m(...)`, first evaluates `e` to obtain the class or object whose method is being called. Then the expressions for the arguments are evaluated to obtain *actual parameter* values; this evaluation happens left to right. Next an *activation record* is created for the call and pushed onto the stack; the activation record contains room for the formal parameters of the method (as discussed earlier, the formals are the variables declared in the method header) and any other local storage the method requires. Then the actual parameters are assigned to the formals; this kind of parameter passing is called *call by value*. Finally, control is *dispatched* to the called method `e.m`; Section 2.5 discusses how this works.

Just as was the case for assignment to variables, if an actual parameter value is a reference to an object, that reference is assigned to the formal. This means that the called procedure shares objects with its caller. Furthermore, if

Figure 2.3 Method call



these objects are mutable, and the called procedure changes their state, these changes are visible to the caller when it returns.

For example, suppose the `Arrays` class mentioned earlier contained a method, `multiples`, that multiplies each element of its array argument `a` by its multiplier argument `m`:

```
public static void multiples (int [ ] a, int m) {
   if (a == null) return;
   for (int i = 0; i < a.length; i++) a[i] = a[i]*m;
}
```

This method works on any size array; it uses `a.length` to determine the length of the array. Figure 2.3 shows what happens when the method is called by the following code:

```
int [ ] b = {1,3,5,7,9};
Arrays.multiples(b, 2);
```

Figure 2.3a shows the situation just before the call. Figure 2.3b shows the situation just after the call has occurred; the stack now contains the activation record for the call, and the formals have been initialized to contain the actuals. Thus, the formal `a` of `Arrays.multiples` refers to the same array as `b` does. Finally, Figure 2.3c shows the situation just after `Arrays.multiples` returns. At this point, the activation record created for the call has been discarded. However, the argument array has been modified, and this modification is visible to the caller through variable `b`.

In a call `e.m` in which `e` is supposed to evaluate to an object, it is possible that `e` might instead evaluate to `null` and thus not refer to any object. If this happens, the call is not made, but instead the `NullPointerException` is raised (exceptions are discussed in Chapter 4).

2.4 Type Checking

Java is a *strongly typed* language, which means that the Java compiler checks the code to ensure that every assignment and every call is type correct. If a type error is discovered, compilation fails with an error message.

Type checking depends on the fact that every variable declaration gives the type of the variable, and the header of every method and constructor defines its *signature*: the types of its arguments and results (and also the types of any exceptions it throws). This information allows the compiler to deduce an *apparent type* for any expression. And this deduction then allows it to determine the legality of an assignment.

For example, consider

```
int y = 7;
int z = 3;
int x = Num.gcd (z, y);
```

When the compiler processes the call to `Num.gcd`, it knows that `Num.gcd` requires two integer arguments, and it also knows that expressions `z` and `y` are both of type int. Therefore, it knows the call of `gcd` is legal. Furthermore, it knows that `gcd` returns an int, and therefore it knows that the assignment to `x` is legal.

Java has an important property: legal Java programs (that is, those accepted by the compiler) are guaranteed to be *type safe*. This means that there cannot be any type errors when the program runs: it is not possible for the program to manipulate data belonging to one type as if it belonged to a different type. Type safety is achieved by three mechanisms: compile-time type checking, automatic storage management, and array bounds checking. Type safety is summarized in Sidebar 2.3.

2.4.1 Type Hierarchy

Java types are organized into a *hierarchy* in which a type can have a number of *supertypes*; we say the type is a *subtype* of each of its *supertypes*. (Other texts may use the term *superclass* [*subclass*] to mean supertype [subtype]; in addition, some texts say a type *extends* another type to mean it is a subtype of the other type.) Type hierarchy provides a way of abstracting from the

Sidebar 2.3 Type Safety

- One important difference between Java and C and C++ is that Java provides type safety. This is accomplished by three mechanisms:
  - Java is a strongly typed language. This means that type errors such as using a pointer as an integer are detected by the compiler.
  - Java provides automatic storage management for all objects. In C and C++, programs manage storage for objects in the heap explicitly. Explicit management is a major source of errors such as *dangling references*, in which storage is deallocated while a program still refers to it.
  - Java checks all array accesses to ensure they are within bounds.
- These techniques ensure that type mismatches cannot occur at runtime. In this way an important source of errors is eliminated from your code.

differences among subtypes to their common behavior, which is captured by their supertype.

The subtype relation is transitive: if `R` is a subtype of `S`, and `S` is a subtype of `T`, then `R` is a subtype of `T`. The relation is also reflexive: type `S` is a subtype of itself.

If `S` is a subtype of `T`, its objects are intended to be usable in any context that expects to use objects belonging to `T`. For `S` objects to be usable, they must have all the methods that `T` objects have; this requirement is enforced by the Java compiler. In addition, all the method calls must behave the same way on `S` and `T` objects; this requirement is *not* enforced by Java, nor could it be, since it requires processing beyond the abilities of a compiler (it requires proving that the two programs behave in the same way). We will discuss this requirement in greater detail in Chapter 7 when we discuss how to define sub- and supertypes. For now, you will only make use of predefined type hierarchies, and you can assume that subtypes are defined properly.

The special type `Object` is at the top of the type hierarchy in Java; all object types, including `String` and array types, are subtypes of this type. This means all objects have certain methods—namely, the ones specified for `Object`. For example, `Object` methods include `equals` and `toString`, with the following headers:

```
boolean equals (Object o)
String toString ( )
```

`Object` and its methods will be discussed further in Chapter 5.

Since objects of a subtype behave like those of a supertype, it makes sense to allow them to be referred to by a variable whose declared type is a supertype. This usage is permitted by Java: an assignment `v = e` is legal if the type of `e` is a subtype of the type of `v`. For example, the following is legal:

```
Object o1 = a;
Object o2 = s;
```

Here `a` is an array, and `s` is a string, as shown in Figure 2.2.

An implication of the assignment rule is that the *actual type* of an object obtained by evaluating an expression is a subtype of the apparent type of the expression deduced by the compiler using declarations. For example, the apparent type of `o2` is `Object`, but its actual type is `String`.

Type checking is always done using the apparent type. This means, for example, that any method calls made using the object will be determined to be legal based on the apparent type. Therefore only `Object` methods like `equals` can be called on `o2`; string methods like `length` (which returns a count of the number of characters in the string) cannot be called:

```
if (o2.equals("abc")) // legal
if (o2.length( )) // illegal
```

Furthermore, the following is illegal:

```
s = o2; // illegal
```

because the apparent type of `o2` is not a subtype of `String`. Compilation will fail when the program contains illegal code as in these examples.

Sometimes a program needs to determine the actual type of an object at runtime, for example, so that a method not provided by the apparent type can be called. This can be done by *casting*. For example,

```
if ((String) o2.length( )) // legal
s = (String) o2; // legal
```

The use of a `cast` causes a check to occur at runtime; if the check succeeds, the indicated computation is allowed, and otherwise, the `ClassCastException` will be raised. In the example, the casts check whether `o2`'s actual type is the